

CPU Scheduling Simulator

Performance Analysis and Algorithm Comparison

Ryan Kurz
CS 3502: Operating Systems
Kennesaw State University

July 23, 2026

Abstract

This project extends a Windows Forms CPU scheduling simulator by adding Shortest Remaining Time First (SRTF) and Highest Response Ratio Next (HRRN). The simulator calculates average waiting time, average turnaround time, average response time, CPU utilization, and throughput. Six algorithms were tested using small, medium, and large workloads to compare efficiency, responsiveness, and fairness. The simulator was also extended with CSV result export and interface documentation for the two new algorithms.

Contents

1	Introduction	2
2	Simulator Implementation	2
2.1	Starter Application	2
2.2	Shortest Remaining Time First	3
2.3	Highest Response Ratio Next	4
2.4	Performance Metrics and CSV Export	5
3	Performance Metrics	6
3.1	Average Waiting Time	6
3.2	Average Turnaround Time	6
3.3	Average Response Time	6
3.4	CPU Utilization	6
3.5	Throughput	7
4	Testing Method	7

5	Results	7
5.1	Small Workload	7
5.2	Medium Workload	8
5.3	Large Workload	8
5.4	Performance Trends	9
6	Analysis and Trade-Offs	10
7	Idle CPU Validation	11
8	Recommendation	11
9	Future Improvements	12
10	Conclusion	12

1 Introduction

CPU scheduling determines which ready process receives access to the processor. Different scheduling algorithms emphasize different goals, including low waiting time, fast response, fairness, and protection from starvation.

The provided simulator already included First Come First Serve (FCFS), Shortest Job First (SJF), Priority Scheduling, and Round Robin. This project added Shortest Remaining Time First and Highest Response Ratio Next. The interface was also extended to calculate performance metrics, display response time, and export results to CSV files.

2 Simulator Implementation

2.1 Starter Application

The project uses C# and Windows Forms. Process information is entered through a table containing a process ID, burst time, priority, and arrival time. Each algorithm produces a result containing the start time, finish time, waiting time, turnaround time, and response time.

The original interface was kept and extended rather than replaced. Two new algorithm buttons were added to the existing flow-layout panel. The Welcome and About pages were updated to list all six algorithms, and the Results page was expanded to display the required performance metrics.

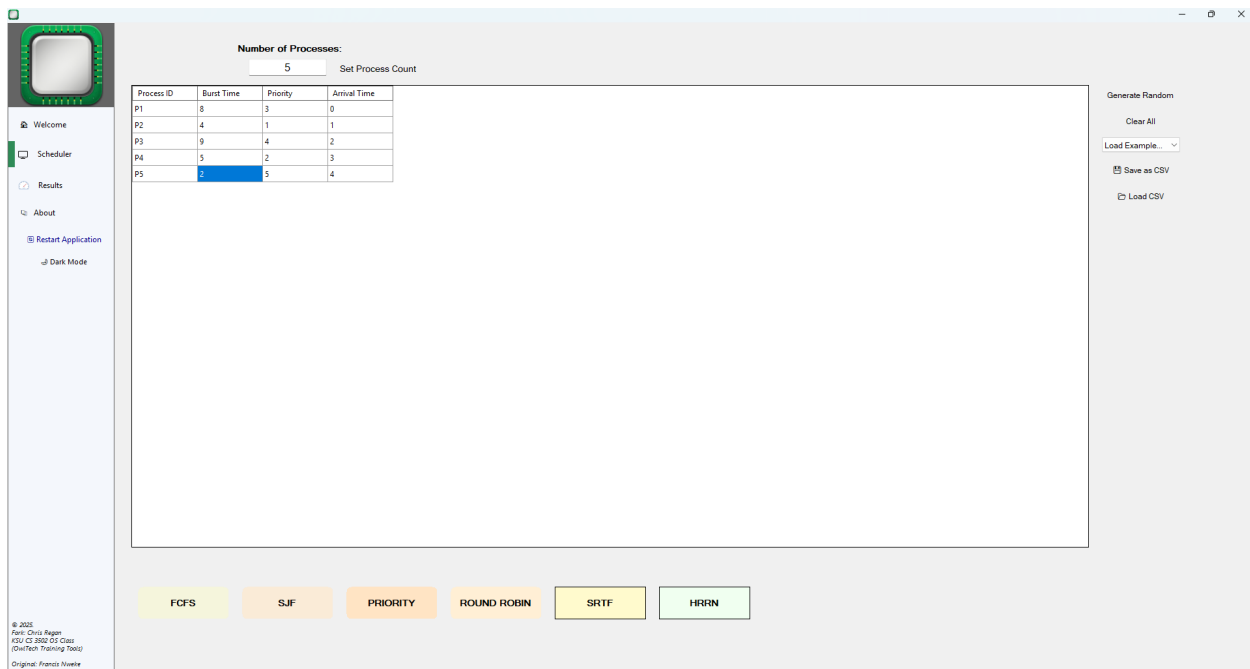


Figure 1: Scheduler page showing all six available algorithms.

2.2 Shortest Remaining Time First

Shortest Remaining Time First is the preemptive version of Shortest Job First. At each unit of simulated time, the scheduler selects the available process with the smallest remaining burst time. A newly arriving shorter process can preempt the currently running process.

SRTF can reduce average waiting and turnaround time, especially when short processes arrive after longer processes. Its main disadvantage is that long processes may experience starvation if short processes continue to arrive.

The main implementation steps were:

1. Store the remaining burst time for every process.
2. At each time unit, collect all arrived processes that are not finished.
3. Select the process with the smallest remaining time.
4. Record its first start time if it has not run before.
5. Execute it for one time unit.
6. When it finishes, calculate finish, turnaround, and waiting times.

Listing 1: Core SRTF selection logic

```
1 var availableProcesses = processes
2   .Where(p => p.ArrivalTime <= currentTime &&
3             remainingTimes[p.ProcessID] > 0)
4   .OrderBy(p => remainingTimes[p.ProcessID])
5   .ThenBy(p => p.ArrivalTime)
6   .ThenBy(p => p.ProcessID)
7   .ToList();
8
9 var currentProcess = availableProcesses.First();
10 remainingTimes[currentProcess.ProcessID]--;
11 currentTime++;
```

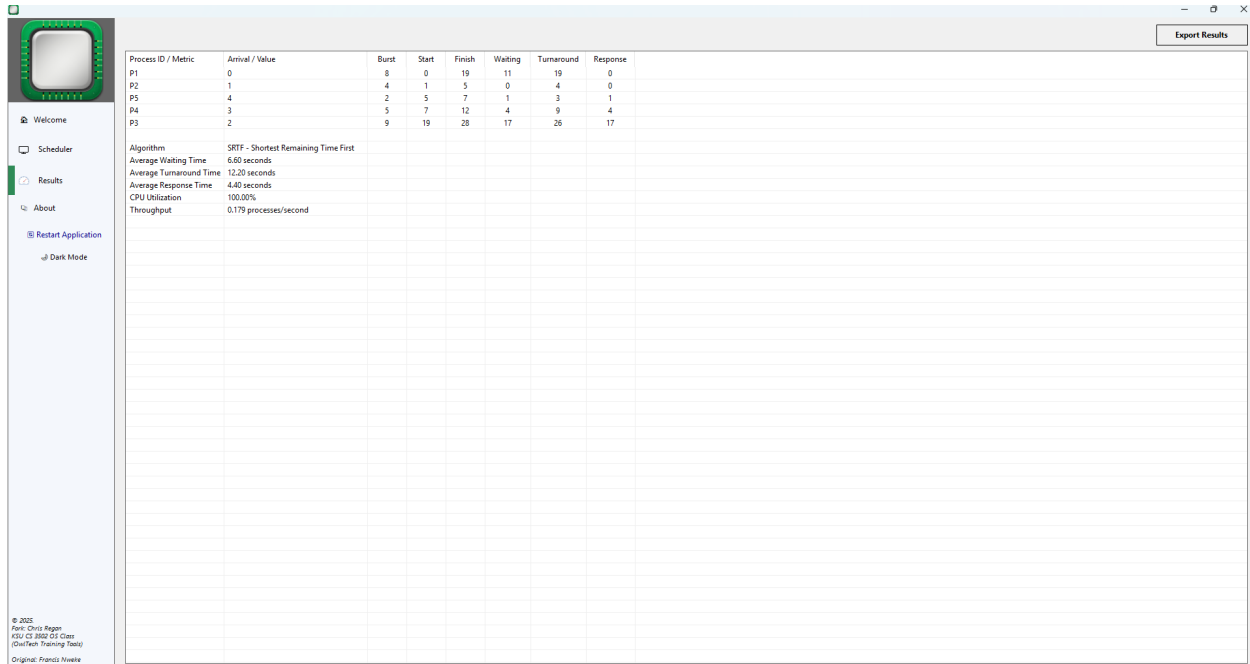


Figure 2: SRTF results for the small workload.

2.3 Highest Response Ratio Next

Highest Response Ratio Next is a non-preemptive scheduling algorithm. It selects the available process with the highest response ratio:

$$R = \frac{W + B}{B} \quad (1)$$

where W is waiting time and B is burst time.

This formula favors short jobs while gradually increasing the priority of processes that have waited for a long time. HRRN therefore provides better starvation protection than basic SJF.

The main implementation steps were:

1. Collect all processes that have arrived by the current time.
2. Calculate the response ratio for each available process.
3. Select the process with the highest ratio.
4. Run the selected process to completion.
5. Calculate its waiting and turnaround times.

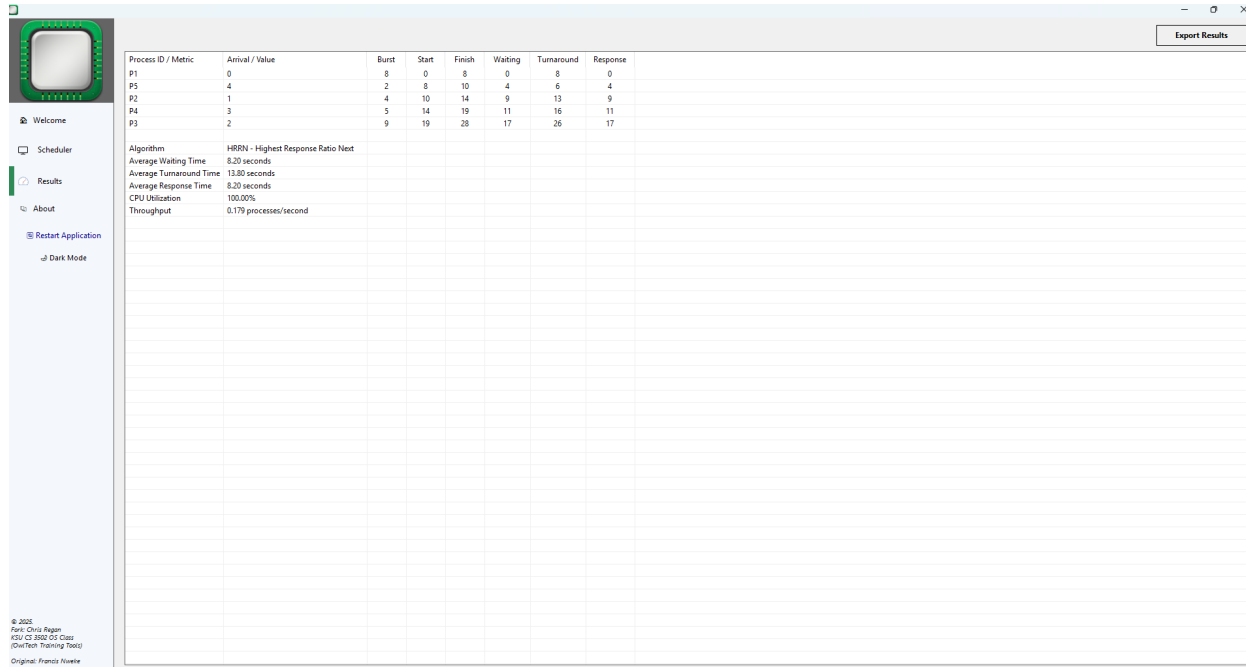
Listing 2: Core HRRN selection logic

```
1 var nextProcess = availableProcesses
2   .OrderByDescending(p =>
```

```

3      ((double)(currentTime - p.ArrivalTime) + p.BurstTime)
4      / p.BurstTime)
5      .ThenBy(p => p.ArrivalTime)
6      .ThenBy(p => p.ProcessID)
7      .First();

```



The screenshot shows a software application window titled "HRRN" with a sidebar on the left containing navigation links: Welcome, Scheduler, Results (selected), and About. Below these are links for "Restart Application" and "Dark Mode". The main area displays a table of process metrics and summary statistics.

Process ID / Metric	Arrival / Value	Burst	Start	Finish	Waiting	Turnaround	Response
P1	0	8	0	8	0	8	0
P5	4	2	9	10	4	6	4
P2	1	4	10	14	9	13	9
P4	3	5	14	19	11	16	11
P3	2	9	19	28	17	26	17
Algorithm	HRRN - Highest Response Ratio Next						
Average Waiting Time	8.20 seconds						
Average Turnaround Time	13.80 seconds						
Average Response Time	8.20 seconds						
CPU Utilization	100.00%						
Throughput	0.179 processes/second						

At the bottom left of the window, there is a copyright notice: "© 2021, Erik Chin, Regan KSV, CE 3592 OS Class (OwTech Training Tools), Original: Francis Nweke". An "Export Results" button is located in the top right corner of the main table area.

Figure 3: HRRN results for the small workload.

2.4 Performance Metrics and CSV Export

The Results page was expanded to show one row per process and a separate set of summary rows. The displayed metrics include average waiting time, average turnaround time, average response time, CPU utilization, and throughput.

An Export Results button was also added. It writes the current process rows, algorithm name, and metric rows to a CSV file. This made it possible to collect results consistently for all workloads and algorithms.

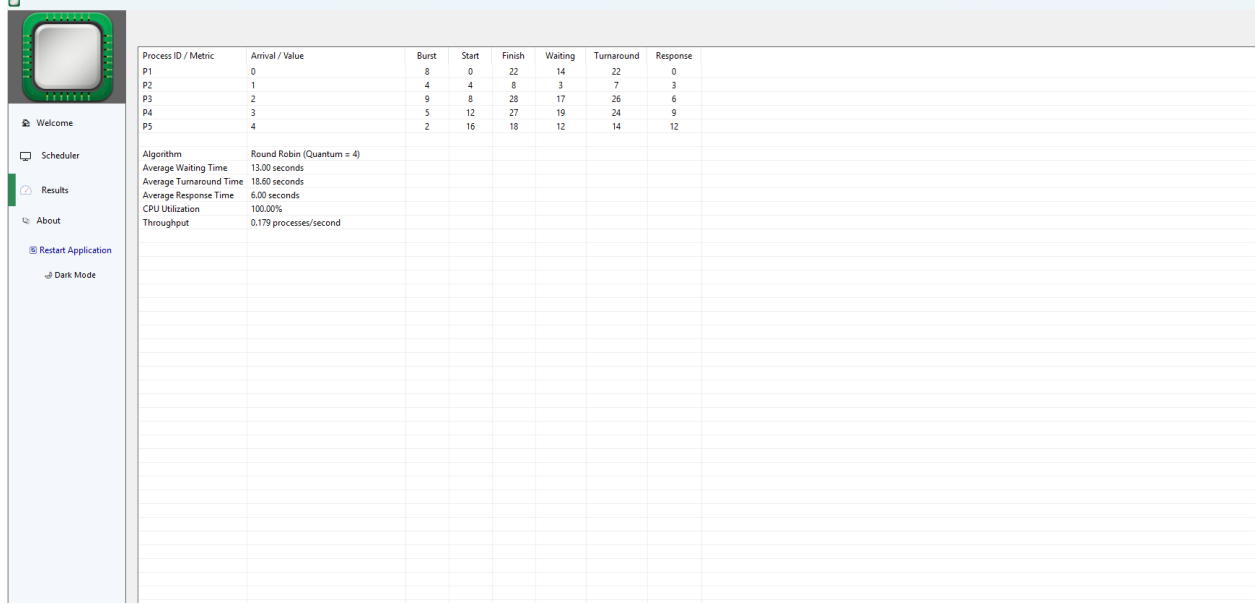


Figure 4: Round Robin results with the Export Results button visible.

3 Performance Metrics

The following metrics were calculated for every algorithm.

3.1 Average Waiting Time

$$AWT = \frac{\sum_{i=1}^n W_i}{n} \quad (2)$$

3.2 Average Turnaround Time

$$ATT = \frac{\sum_{i=1}^n T_i}{n} \quad (3)$$

3.3 Average Response Time

$$ART = \frac{\sum_{i=1}^n (S_i - A_i)}{n} \quad (4)$$

where S_i is the first start time and A_i is the arrival time.

3.4 CPU Utilization

$$CPU \text{ Utilization} = \frac{\text{Total Burst Time}}{\text{Total Elapsed Time}} \times 100 \quad (5)$$

3.5 Throughput

$$Throughput = \frac{\text{Number of Completed Processes}}{\text{Total Elapsed Time}} \quad (6)$$

4 Testing Method

Three main workload sizes were tested:

- Small workload: 5 processes
- Medium workload: 20 processes
- Large workload: 100 processes

All six algorithms were tested using the same process data for each workload. Round Robin used a time quantum of 4. The small workload was manually defined, the medium workload used mixed burst and arrival times, and the large workload used heavier burst times. An additional idle-gap workload was used to verify that CPU utilization correctly decreases when no process is ready.

5 Results

5.1 Small Workload

Table 1: Small workload performance

Algorithm	Avg. Wait	Avg. Turnaround	Avg. Response
FCFS	11.40	17.00	11.40
SJF	8.20	13.80	8.20
Priority	10.20	15.80	10.20
Round Robin	13.00	18.60	6.00
SRTF	6.60	12.20	4.40
HRRN	8.20	13.80	8.20

For the small workload, SRTF produced the lowest average waiting time, turnaround time, and response time.

5.2 Medium Workload

Table 2: Medium workload performance

Algorithm	Avg. Wait	Avg. Turnaround	Avg. Response
FCFS	104.15	115.20	104.15
SJF	74.65	85.70	74.65
Priority	103.80	114.85	103.80
Round Robin	161.35	172.40	35.00
SRTF	73.15	84.20	68.85
HRRN	78.90	89.95	78.90

For the medium workload, SRTF again produced the lowest waiting and turnaround times. Round Robin produced the lowest response time but substantially higher waiting and turnaround times.

5.3 Large Workload

Table 3: Large workload performance

Algorithm	Avg. Wait	Avg. Turnaround	Avg. Response
FCFS	1002.27	1022.64	1002.27
SJF	831.02	851.39	831.02
Priority	1008.06	1028.43	1008.06
Round Robin	1771.49	1791.86	195.58
SRTF	831.02	851.39	831.02
HRRN	839.56	859.93	839.56

For the large workload, SJF and SRTF tied for the lowest waiting and turnaround times. Round Robin again produced the lowest response time but the highest waiting and turnaround times.

5.4 Performance Trends

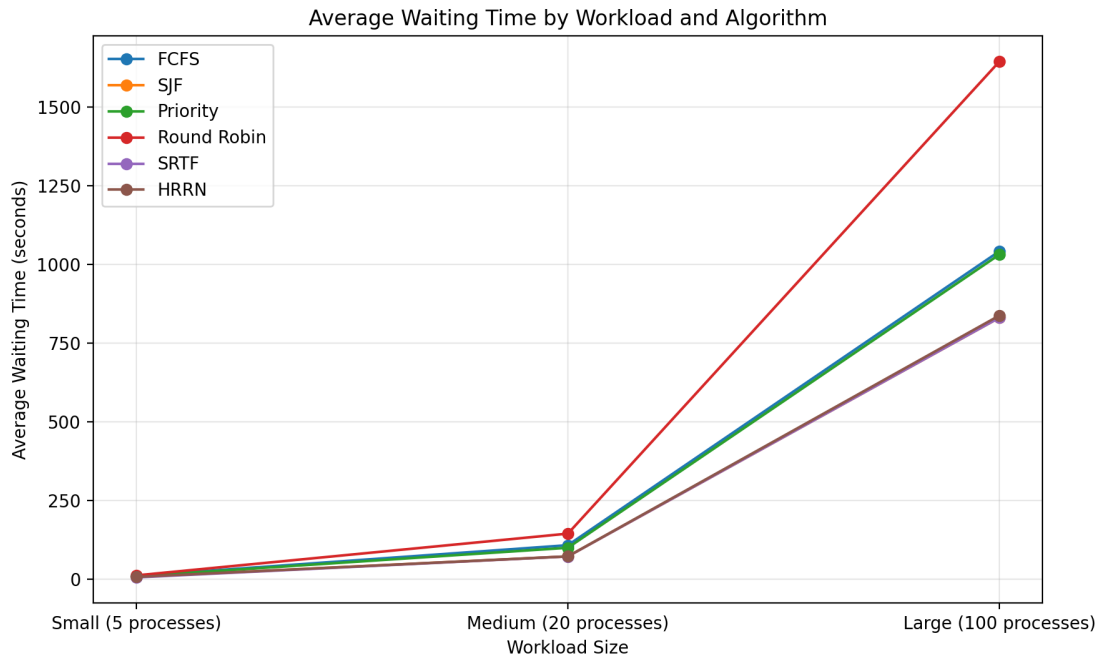


Figure 5: Average waiting time by workload and algorithm.

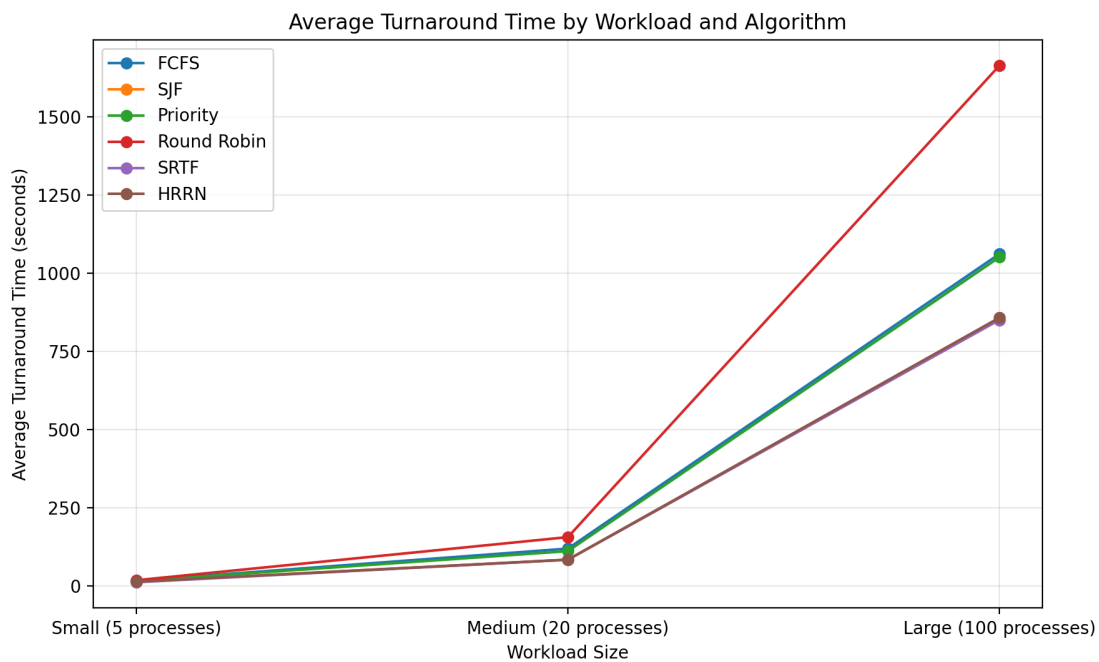


Figure 6: Average turnaround time by workload and algorithm.

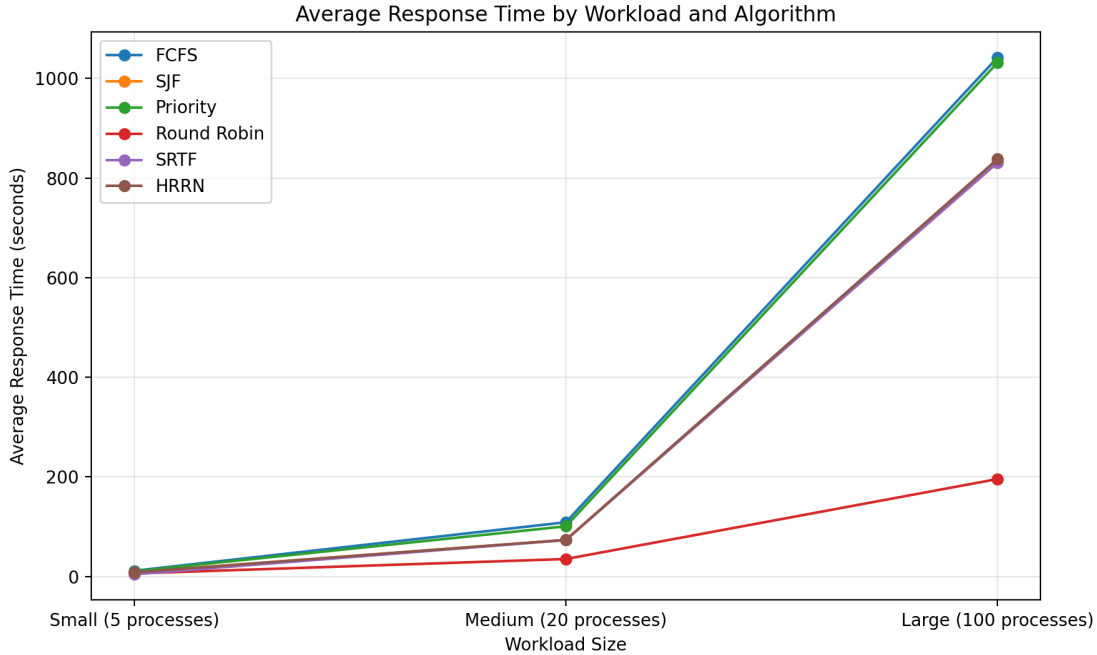


Figure 7: Average response time by workload and algorithm.

The graphs show that waiting and turnaround times rise significantly as the number of processes increases. SJF, SRTF, and HRRN scale better than FCFS, Priority, and Round Robin for waiting and turnaround time. Round Robin scales poorly for completion-related metrics but remains strong for initial response.

6 Analysis and Trade-Offs

SRTF produced the best or tied-best average waiting and turnaround times across the tested workloads. This occurred because it always favored the process with the least remaining work. However, this strategy can delay long processes when short jobs continue to arrive.

SJF performed similarly to SRTF on the large workload because the generated arrival pattern did not create enough useful preemption opportunities to give SRTF an advantage. SJF is simpler because it is non-preemptive, but it can also starve long processes.

HRRN stayed close to SJF while providing better starvation protection. Its response-ratio formula increases a process's chance of selection as its waiting time grows. This makes it a reasonable compromise between efficiency and fairness.

Round Robin produced the best response time for the medium and large workloads. Every process received an early time slice, which improves responsiveness. However, repeated queue cycling increased average waiting and turnaround times. In a real operating system, context-switch overhead would make this trade-off even more significant.

FCFS was simple and predictable but suffered from the convoy effect when long processes appeared before short ones. Priority Scheduling depended heavily on the assigned priorities and can starve low-priority processes without aging.

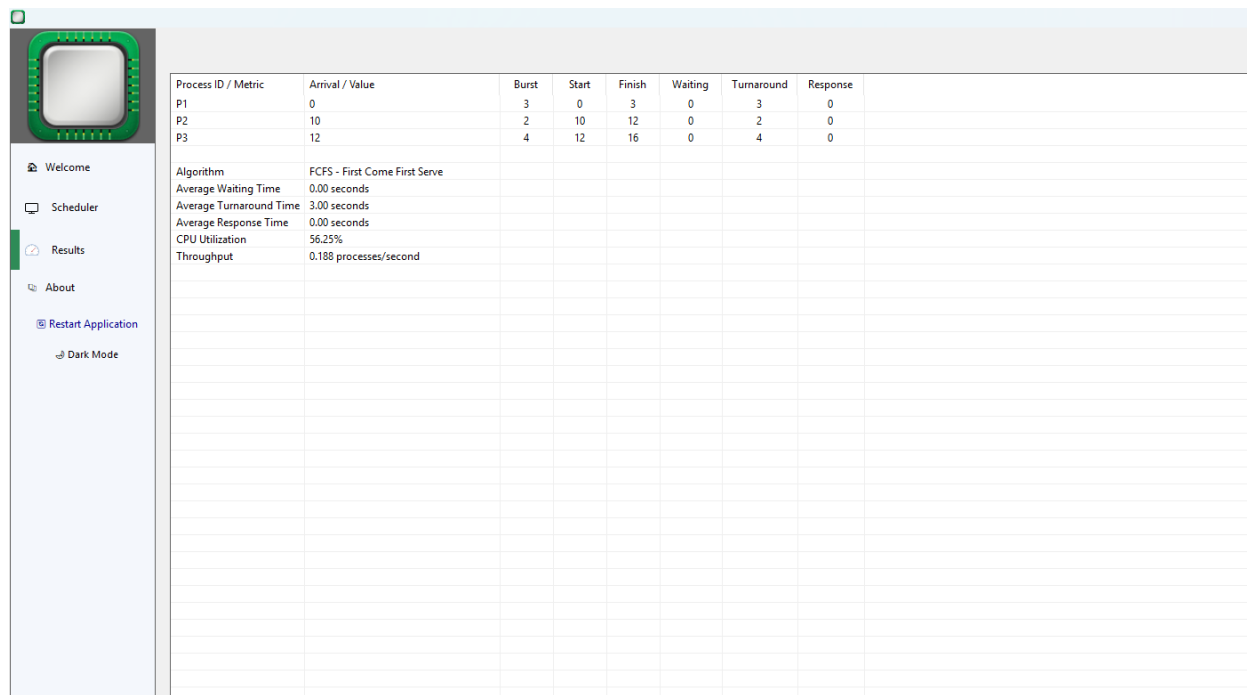
CPU utilization and throughput were the same for algorithms within each main workload because they processed the same total burst time without idle periods. The meaningful differences were therefore waiting, turnaround, and response times.

7 Idle CPU Validation

An additional test used three processes with arrival times of 0, 10, and 12. The CPU was idle between time 3 and time 10. The simulator reported CPU utilization of 56.25 percent, matching the manual calculation:

$$\frac{3 + 2 + 4}{16} \times 100 = 56.25\% \quad (7)$$

This confirmed that idle time was included correctly in the utilization calculation.



Process ID / Metric	Arrival / Value	Burst	Start	Finish	Waiting	Turnaround	Response
P1	0	3	0	3	0	3	0
P2	10	2	10	12	0	2	0
P3	12	4	12	16	0	4	0
Algorithm	FCFS - First Come First Serve						
Average Waiting Time	0.00 seconds						
Average Turnaround Time	3.00 seconds						
Average Response Time	0.00 seconds						
CPU Utilization	56.25%						
Throughput	0.188 processes/second						

Figure 8: Idle-gap validation showing 56.25 percent CPU utilization.

8 Recommendation

For workloads where minimizing waiting and turnaround time is the main goal, OwlTech should use SRTF. It performed best or near best across the tested workloads.

For interactive systems where fast initial response and fairness are more important, Round Robin is preferable. However, its time quantum must be chosen carefully because a small quantum increases queue cycling and context switching.

HRRN is recommended when starvation prevention is important but the system still needs performance similar to SJF. It provides a strong balance between efficiency and fairness without requiring preemption.

9 Future Improvements

Possible future improvements include:

- Adding a Gantt chart timeline
- Supporting Multilevel Feedback Queue scheduling
- Automatically running every algorithm on one workload
- Exporting a combined comparison CSV
- Measuring actual simulator execution time
- Adding context-switch overhead
- Adding priority aging
- Letting the user compare algorithms side by side in the interface

10 Conclusion

The project successfully added SRTF and HRRN to the provided CPU scheduling simulator. It also added response time, CPU utilization, throughput, CSV export, updated interface documentation, and expanded result display functionality.

Testing showed that no single algorithm is ideal for every workload. SRTF provided strong waiting and turnaround performance, Round Robin provided fast response, and HRRN offered a useful balance between efficiency and fairness. The final recommendation depends on whether the target system values completion time, responsiveness, or starvation prevention most strongly.

References

- [1] CS 3502 Project 2: CPU Scheduling Simulator, Kennesaw State University, Department of Computer Science, 2026.
- [2] CPU-Simulator using Windows Forms, starter repository maintained by Chris Regan, original creator Francis.
- [3] J. Bell, “CPU Scheduling,” Operating Systems Course Notes, University of Illinois Chicago.